

Jenkins CI/CD with Terraform on AWS

Introduction

This project demonstrates a **CI/CD pipeline using Jenkins** to provision and deploy infrastructure on **AWS using Terraform**. The pipeline automates **infrastructure provisioning**, application deployment, and management.

Steps to Set Up Jenkins CI/CD with Terraform on AWS

1. Prerequisites

- AWS Account
- EC2 instance for Jenkins (Ubuntu preferred)
- IAM role and user for Terraform with required permissions
- S3 bucket and DynamoDB table for Terraform state management
- Jenkins installed on the EC2 instance
- Terraform installed on the Jenkins server

2. Setup Jenkins on AWS EC2

Launch an EC2 instance and install Jenkins: `sudo apt update && sudo apt install -y openjdk-11-jdk`
`wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo apt-key add -`
`sudo sh -c 'echo deb http://pkg.jenkins.io/debian-stable binary/ > /etc/apt/sources.list.d/jenkins.list'`
`sudo apt update`
`sudo apt install -y jenkins`
`sudo systemctl start jenkins`
`sudo systemctl enable jenkins`

Open port 8080 in AWS Security Group and get the admin password: `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`

- **Install necessary Jenkins plugins:**
 - Terraform
 - AWS Credentials
 - Git
 - Pipeline

3. Configure Terraform for AWS

Install Terraform on Jenkins EC2: `wget https://releases.hashicorp.com/terraform/1.7.0/terraform_1.7.0_linux_amd64.zip`
`unzip terraform_1.7.0_linux_amd64.zip`
`sudo mv terraform /usr/local/bin/`

terraform -v

- **Create an IAM User** in AWS with the following permissions: ○ AmazonS3FullAccess (for Terraform state)

- AmazonDynamoDBFullAccess (for state locking)

- AdministratorAccess (for deploying resources)

- **Store AWS Credentials in Jenkins:** ○ Go to **Jenkins > Manage Jenkins > Manage Credentials**

- Add AWS Access Key and Secret Key

4. Create Terraform Files

- **main.tf:** Define AWS infrastructure (e.g., EC2, S3, RDS)

- **variables.tf:** Store variable values

- **outputs.tf:** Define output values

- **provider.tf:** Configure AWS provider

Example main.tf for EC2:

```
hcl
```

```
provider "aws" {
```

```
region = "us-east-1"
```

```
}
```

```
resource "aws_instance" "jenkins_server" {
```

```
ami = "ami-0c55b159cbfafa1f0"
```

```
instance_type = "t2.micro"
```

```
key_name = "my-key"
```

```
tags = {
```

```
Name = "Jenkins-Server"
```

```
}
```

```
}
```

Initialize Terraform: terraform init

terraform validate

5. Set Up Jenkins Pipeline for Terraform

- Create a **Jenkins Pipeline Job**

- Add the following Jenkinsfile in the repository:

```

groovy
pipeline {
    agent any

    environment {
        AWS_ACCESS_KEY_ID = credentials('AWS_ACCESS_KEY_ID')
        AWS_SECRET_ACCESS_KEY = credentials('AWS_SECRET_ACCESS_KEY')
    }

    stages {
        stage('Checkout Code') {
            steps {
                git 'https://github.com/your-repo/terraform-aws.git'
            }
        }

        stage('Initialize Terraform') {
            steps {
                sh 'terraform init'
            }
        }

        stage('Plan Terraform') {
            steps {
                sh 'terraform plan'
            }
        }

        stage('Apply Terraform') {
            steps {
                sh 'terraform apply -auto-approve'
            }
        }
    }
}

```

6. Run the CI/CD Pipeline

- Go to **Jenkins Dashboard** → **Select the Pipeline Job**
- Click on **Build Now**
- Monitor logs for successful deployment

7. Verify AWS Resources

- Go to AWS Console
- Check if EC2, S3, RDS, or other services were provisioned

8. Destroy Infrastructure (Optional)

If you want to delete the deployed infrastructure: terraform destroy -auto-approve

Conclusion

This setup automates **AWS infrastructure deployment using Jenkins and Terraform**. It ensures that infrastructure is provisioned and managed efficiently as part of a CI/CD pipeline.